



Universidade do Minho

Licenciatura em Engenharia Informática - LEI

Computação Gráfica

Trabalho Prático - Fase 1

Grupo 14

A107324 - David José Barbosa Alves

A107368 - Diogo Malheiro Pais

A107377 - João Pedro Ferreira Gomes

A107332 - Pedro Afonso Quesado Pacheco Pereira

1. Introdução

O presente relatório descreve o desenvolvimento da primeira fase do projeto prático da unidade curricular de Computação Gráfica. O objetivo central desta etapa inicial é a criação das bases de um motor de visualização 3D, assente num sistema de *scene graph*, e de uma ferramenta auxiliar para a geração de modelos geométricos.

O trabalho realizado dividiu-se em duas aplicações distintas e complementares:

1. **Generator**: Uma aplicação de linha de comandos dedicada à computação algorítmica de formas geométricas primitivas. Nesta fase, o foco recaiu sobre a geração de planos, cubos, esferas e cones. Para cada uma destas figuras, o programa calcula as coordenadas dos vértices necessários para a sua representação através de triângulos, exportando os dados para um formato de ficheiro próprio (extensão *.3d*).

Figura	Comando Exemplo	Parâmetros Detalhados
Plane	<code>./generator plane 2 2 plane.3d</code>	<comprimento><divisões> <ficheiro>
Box	<code>./generator box 2 3 box.3d</code>	<dimensão> <divisões> <ficheiro>
Sphere	<code>./generator sphere 1 10 10 sphere.3d</code>	<raio> <slices> <stacks> <ficheiro>
Cone	<code>./generator cone 1 2 4 3 cone.3d</code>	<raio> <altura> <slices> <stacks> <ficheiro>

Tabela 1 - Comandos de invocação do Generator para as várias primitivas geométricas.

2. **Engine:** Um motor gráfico que utiliza a API OpenGL para a renderização das cenas. Esta aplicação é responsável por processar ficheiros de configuração em formato XML, que definem os parâmetros da câmara (posição, orientação e projeção) e listam os modelos geométricos a serem carregados e visualizados no ecrã.

Nesta fase, a arquitetura do sistema foi desenhada para garantir a leitura eficiente dos dados e a sua correta apresentação visual em *Immediate Mode*, servindo de alicerce para as funcionalidades de transformações geométricas e otimização que serão implementadas nas fases seguintes.

2. O Generator (Algoritmos e Geometria)

A aplicação **Generator** foi desenvolvida como uma ferramenta de linha de comandos independente, cujo propósito é a computação algorítmica de primitivas geométricas. O programa calcula as coordenadas dos vértices necessários para representar superfícies 3D e exporta-as para um ficheiro com extensão **.3d**.

Para garantir a correta renderização e iluminação nas fases seguintes, foi adotada a convenção de orientação de vértices **Counter-Clockwise (CCW)** ou pela **Regra da Mão Direita**, ensinada nas aulas. Esta decisão assegura que a face visível (frente) de cada triângulo está orientada para o exterior do sólido.

2.1 Plano (Plane)

O plano é gerado no plano horizontal XZ, centrado na origem (0,0,0).

- **Abordagem e Decisões:** A superfície é definida por um comprimento de lado L e dividida numa grelha de $N \times N$ quadrados (células). Esta subdivisão é essencial para permitir, futuramente, efeitos de iluminação e texturização mais detalhados.
- **Algoritmo:** Para cada célula da grelha, calculam-se os quatro vértices dos seus cantos. Dado um número de divisões D , o incremento entre vértices é definido por $\Delta = L/D$. As coordenadas x e z variam no intervalo $[-L/2, L/2]$.

- **Geometria:** Cada célula quadrada é decomposta em dois triângulos. Por exemplo, para uma célula com vértices $\{V1, V2, V3, V4\}$, os triângulos gerados são $\{V1, V2, V3\}$ e $\{V1, V3, V4\}$, com as seguintes setas garantindo uma orientação Counter-Clockwise (CCW) para que a face visível esteja voltada para o exterior, tal como se vê na Figura 1

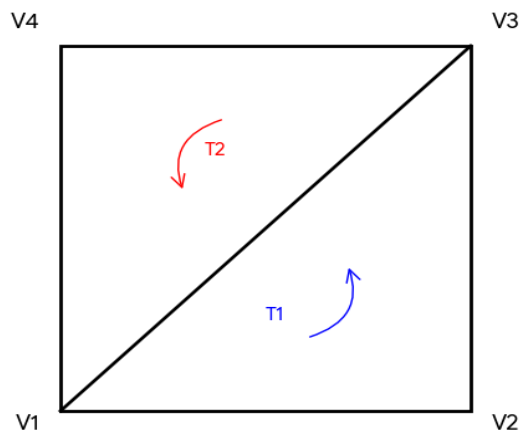


Figura 1: Esquema de uma célula da grelha do plano.

2.2 Cubo (Box)

O cubo é definido pelas suas dimensões (comprimento de aresta) e pelo número de subdivisões por face.

- **Abordagem:** O sólido é construído como um conjunto de seis planos independentes. Cada plano é perpendicular a um dos eixos principais (X, Y ou Z) e deslocado de uma distância $d = \text{dimensão}/2$ em relação à origem.
- **Algoritmo:** É aplicada a lógica de subdivisão do plano a cada uma das seis faces. A principal dificuldade algorítmica reside no ajuste da orientação dos triângulos em cada face, garantindo que todas as normais apontam para o exterior do cubo, respeitando a regra CCW.

2.3 Esfera (Sphere)

A esfera é gerada parametricamente com base num raio r , fatias (*slices*) e camadas (*stacks*).

- **Equações:** Para converter a representação angular em coordenadas cartesianas (x,y,z) , utilizam-se as seguintes equações de coordenadas esféricas:
 - $x = r \cdot \cos(\phi) \cdot \sin(\theta)$
 - $y = r \cdot \sin(\phi)$
 - $z = r \cdot \cos(\phi) \cdot \cos(\theta)$
- **Abordagem:** O ângulo θ (longitude) varia entre 0 e 2π radianos conforme as *slices*, enquanto ϕ (latitude) varia entre $-\pi/2$ e $\pi/2$ conforme as *stacks*. Os triângulos são gerados ligando vértices adjacentes entre camadas consecutivas, formando uma malha fechada.

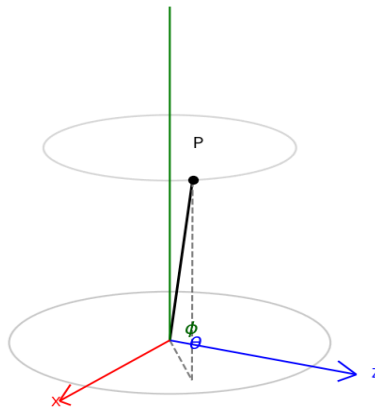


Figura 2: Sistema de coordenadas esféricas utilizado na geração da malha.

2.4 Cone

O cone é gerado com a base assente no plano XZ e o ápice no eixo Y positivo.

- **Base:** Consiste num disco circular subdividido em triângulos. Estes ligam o centro da base $(0,0,0)$ a pontos calculados no perímetro do círculo, utilizando as fórmulas de trigonometria circular $x=r \cdot \cos(\theta)$ e $z=r \cdot \sin(\theta)$.

- **Corpo Lateral:** O corpo é subdividido em camadas (*stacks*) e fatias.
- **Lógica de Subdivisão:** Para garantir a forma cônica, o raio em cada altura h é reduzido linearmente:

$$r_{camada} = r_{base} \cdot ((alturatotal - hatual) / alturatotal)$$

Os triângulos são construídos ligando os vértices da base de cada camada aos vértices da camada superior, convergindo todos para o ápice no topo.

3. Engine

O **Engine** é a aplicação responsável pela leitura de ficheiros de configuração e pela renderização gráfica da cena 3D. O seu funcionamento baseia-se num pipeline que transforma descrições textuais (XML) e dados geométricos brutos (.3d) numa representação visual interativa através da API OpenGL.

3.1. Arquitetura e Fluxo de Execução

Para garantir a eficiência do sistema, a arquitetura do motor foi dividida em três fases principais, assegurando que operações pesadas (como a leitura de ficheiros) sejam realizadas apenas uma vez:

1. **Parsing de Configuração:** No arranque, o motor utiliza a biblioteca **TinyXML** para interpretar o ficheiro XML fornecido. São extraídos os parâmetros da janela, as definições da câmara e a lista de modelos a carregar.
2. **Carregamento de Geometria:** Para cada modelo referenciado, o motor abre o ficheiro **.3d** correspondente, lê o número de vértices e armazena as coordenadas x, y, z em estruturas de dados dinâmicas (como **std::vector**) na memória RAM.
3. **Ciclo de Renderização:** Durante a execução, o motor percorre as estruturas de dados e envia os vértices para a placa gráfica em cada frame.

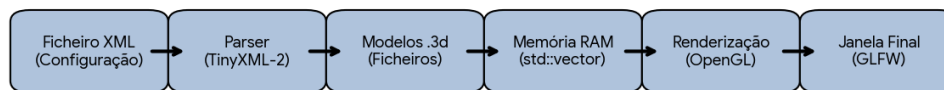


Figura 3: Diagrama de fluxo da arquitetura do Engine.

3.2. Configuração da Câmara e Projeção

A visualização é determinada pelos parâmetros definidos na tag **<camera>** do XML:

- **Posicionamento:** A localização do observador é definida por um ponto no espaço (x,y,z).
- **Orientação:** O motor utiliza a função **gluLookAt** para definir para onde a câmara aponta e qual o vetor de orientação vertical (*Up Vector*).
- **Projeção:** É aplicada uma perspetiva baseada num campo de visão (*Field of View*), definindo também os planos de corte *near* e *far* para limitar o volume de visualização.

3.3. Renderização (Desenho)

Nesta fase inicial do projeto, a renderização dos modelos é efetuada recorrendo ao **Immediate Mode** do OpenGL.

- **Lógica:** Para cada modelo carregado, o motor inicia um bloco de desenho de triângulos (**glBegin(GL_TRIANGLES)**).
- **Processamento de Vértices:** O motor percorre a lista de pontos armazenada e emite cada coordenada através do comando **glVertex3f**.
- **Resultado:** Este método, embora simples, permite validar de forma imediata a integridade da geometria gerada pelo *Generator* e a correta orientação das faces (CCW).

4. Resultados e Testes

Nesta secção, apresentamos a validação das ferramentas desenvolvidas através da execução dos ficheiros de configuração XML fornecidos pela equipa docente. O objetivo foi garantir que a geometria gerada pelo **generator** é lida corretamente pelo **engine** e que a visualização final corresponde às imagens de referência.

4.1. Validação das Primitivas

Foram realizados testes individuais para cada primitiva gerada:

- **Plano e Cubo:** Verificou-se a correta subdivisão das faces e a ausência de faces invertidas, confirmando que a lógica CCW foi aplicada com sucesso.
- **Esfera e Cone:** A utilização de coordenadas esféricas e a redução linear do raio no cone produziram malhas suaves, cuja densidade de triângulos variou conforme o número de *slices* e *stacks* definidos.

4.2. Formato de Armazenamento

Utilizando os comandos de execução do motor, validou-se a capacidade do sistema em processar modelos complexos.

Gestão de Memória: A estrutura do ficheiro **.3d** revelou-se altamente eficiente. Ao armazenar o número total de vértices na primeira linha, o motor consegue realizar a alocação dinâmica de memória de forma imediata, otimizando drasticamente o tempo de carregamento inicial, mesmo em ambientes com múltiplos objetos.

4.2.1 Estrutura Interna do Ficheiro .3d

| Conteúdo do Ficheiro | Descrição Técnica |

| 12345 | **Número total de vértices** (Cabeçalho para alocação de memória) |

| 0.000000 0.000000 0.0000 | **Vértice 1** (Coordenadas cartesianas x,y,z) |

| 1.000000 0.000000 1.0000 | **Vértice 2** (x,y,z) |

| ... | **Vértices seguintes** até perfazer o total indicado |

Leitura Sequencial: O motor lê primeiro o número de vértices (cabeçalho), permitindo a alocação dinâmica imediata de memória através de um **std::vector**.

Simplicidade: Ao contrário de formatos complexos como o **.obj**, este formato proprietário elimina a necessidade de parsing de faces ou texturas nesta fase, tornando o carregamento instantâneo.

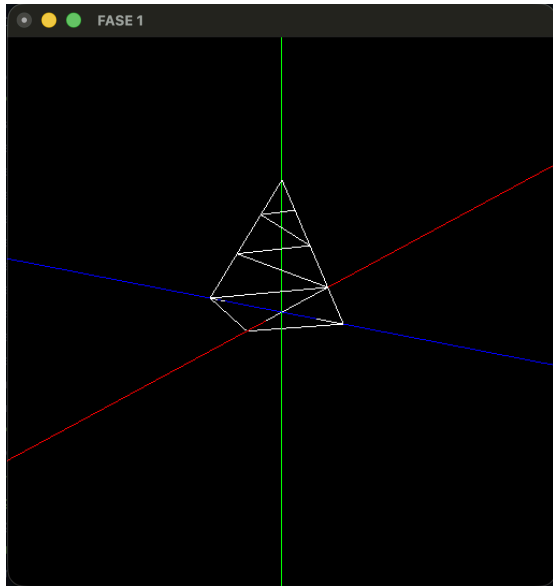


Figura 4: Cone (Gerado)

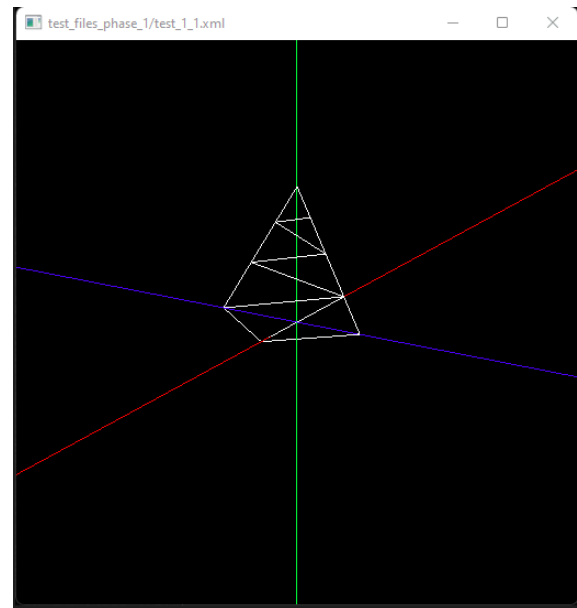


Figura 5: Cone (Exemplo)

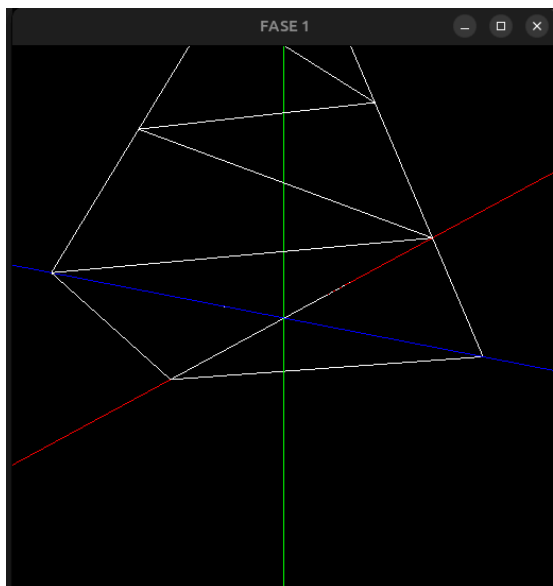


Figura 6: Cone Ampliado (Gerado)

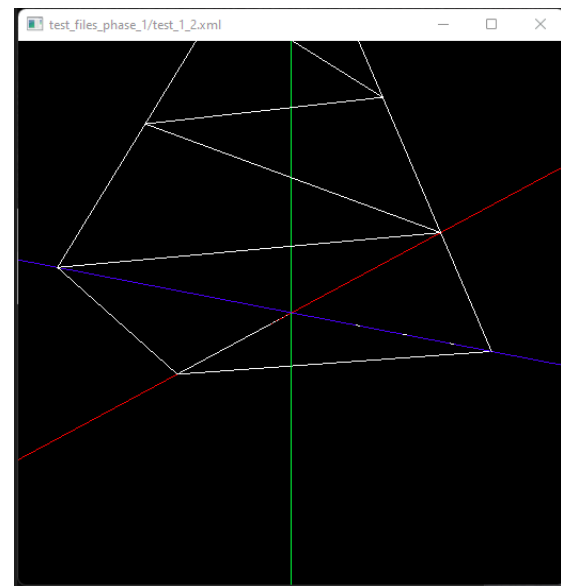


Figura 7: Cone Ampliado (Exemplo)

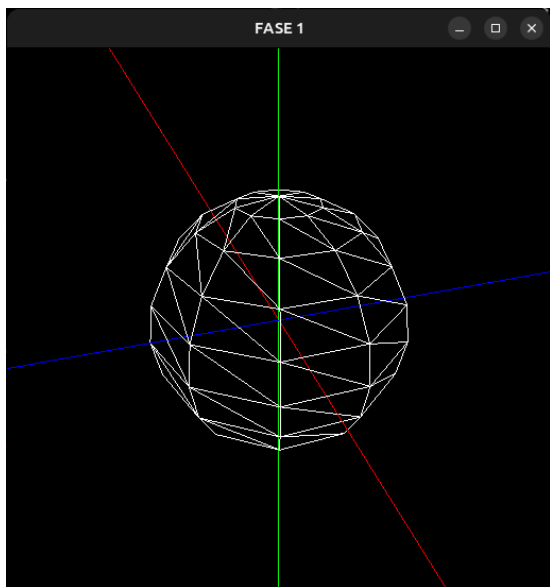


Figura 8: Esfera (Gerado)

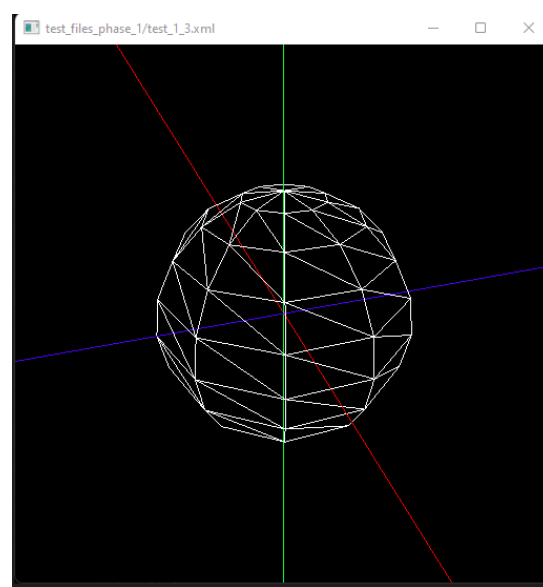


Figura 9: Esfera (Exemplo)

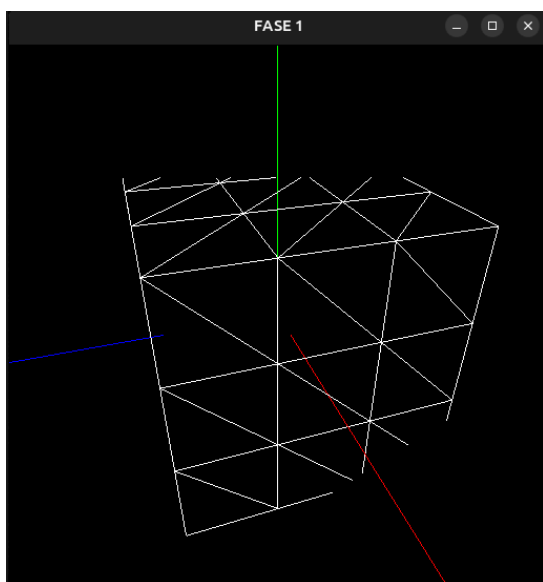


Figura 10: Cubo (Gerado)

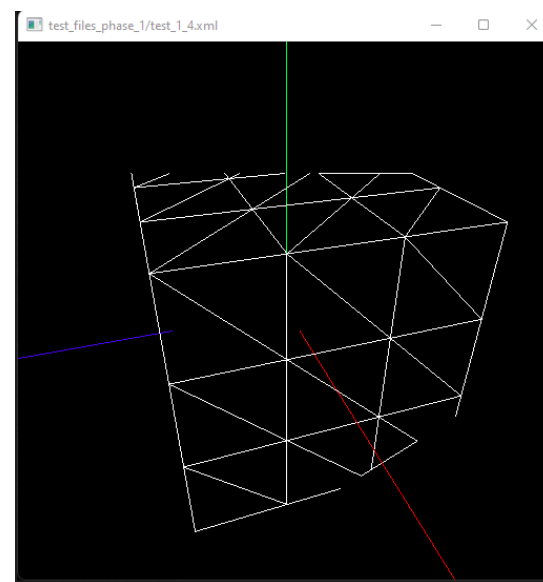


Figura 11: Cubo (Exemplo)

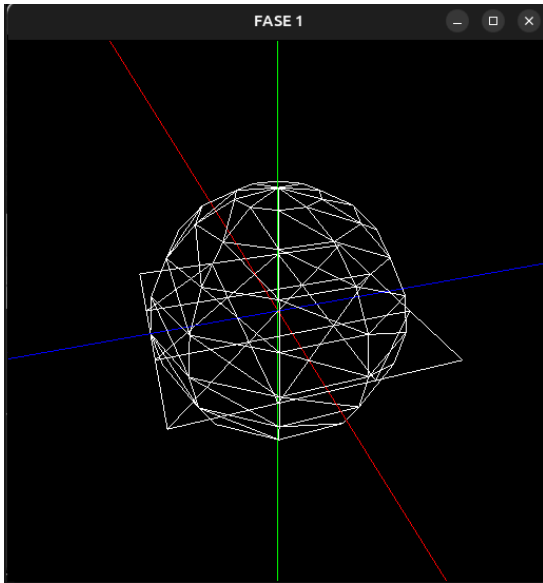


Figura 12: Esfera com Plano (Gerado)

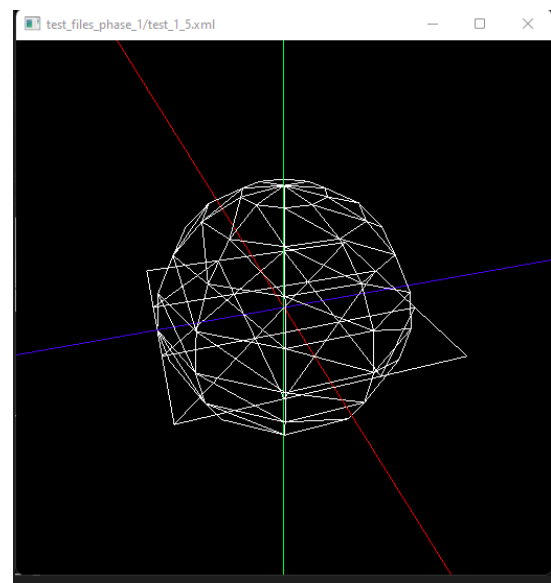


Figura 13: Esfera com Plano (Exemplo)

4. Conclusão

A primeira fase do projeto foi concluída com sucesso, resultando em duas aplicações robustas que comunicam de forma eficaz através de ficheiros de dados proprietários e configurações XML.

Os principais objetivos atingidos foram:

- **Automação Geométrica:** A implementação de algoritmos para a geração de malhas complexas (esfera e cone) a partir de parâmetros simples.
- **Modularidade:** A separação entre o gerador de modelos e o motor de visualização permite que a geometria seja processada *offline*, reduzindo a carga computacional do motor.
- **Arquitetura do Motor:** O desenvolvimento de um sistema capaz de interpretar cenas complexas via XML, estabelecendo uma base sólida para a introdução de hierarquias de transformações na fase seguinte.

Como trabalho futuro para a **Fase 2**, o motor será expandido para suportar transformações geométricas (translação, rotação e escala) e a estruturação da cena numa árvore hierárquica (*Scene Graph*).

5. Referências Bibliográficas

- [1] **OpenGL**. "The Industry's Foundation for High Performance Graphics." Acedido em: Março 2026. [Online]. Disponível em: <https://www.opengl.org/>.
- [2] **FileFormat.Info**. "Wavefront OBJ File Format Summary." Acedido em: Março 2026. [Online]. Disponível em: <https://www.fileformat.info/format/wavefrontobj/egff.htm>.
- [3] **Songho CA**. "OpenGL Sphere - Coordenadas Esféricas." Acedido em: Março 2026. [Online]. Disponível em: https://www.songho.ca/opengl/gl_sphere.html.
- [4] **Songho CA**. "OpenGL Cylinder, Prism & Pipe." Acedido em: Março 2026. [Online]. Disponível em: https://www.songho.ca/opengl/gl_cylinder.html.
- [5] **Wolfram MathWorld**. "Torus - Equações Paramétricas." Acedido em: Março 2026. [Online]. Disponível em: <https://mathworld.wolfram.com/Torus.html>.
- [6] **GLFW**. "An OpenGL library." Acedido em: Fevereiro 2026. [Online]. Disponível em: <https://www.glfw.org>.
- [7] **TinyXML-2**. "GitHub - leethomason/tinyxml2." Acedido em: Março 2026. [Online]. Disponível em: <https://github.com/leethomason/tinyxml2>.
- [8] **Khronos Registry**. "gluLookAt - View Matrix Calculation." Acedido em: Março 2026. [Online]. Disponível em: <https://registry.khronos.org/OpenGL-Refpages/gl2.1/xhtml/gluLookAt.xml>.
- [9] **GLM**. "OpenGL Mathematics Library." Acedido em: Março 2026. [Online]. Disponível em: <https://github.com/g-truc/glm>.